NAAN MUDHALVAN – BIG DATA

Practical Test Cases – Procedures, Programs, Commands and Sample Outputs

This material follows the nine test cases in the uploaded document. The sample records and outputs below are illustrative; your actual output may differ if you use a different dataset or software version.

Sample dataset used in Hive and Spark examples

Create a CSV file named students.csv with the following contents. Use the same values when checking the sample results.

```
id,name,department,year,marks,join_date
1,Asha,CSE,2024,85,2024-06-10
2,Bala,IT,2024,78,2024-07-15
3,Charan,CSE,2025,92,2025-06-12
4,Divya,ECE,2025,68,2025-07-20
5,Elan,IT,2025,88,2025-06-18
```

1. Hive Functions – Demo

Aim

To create a Hive table and demonstrate aggregate, string and date functions.

Procedure

1. Start the Hadoop and Hive services provided by your lab.
2. Create a database and select it.
3. Create a table with columns matching the CSV file.
4. Load the CSV file into the table.
5. Run COUNT, SUM, AVG, MIN, MAX, string and date function queries.
6. Capture each query and its output.

Commands and queries

```
CREATE DATABASE bigdata_demo;
USE bigdata_demo;

CREATE TABLE students (
  id INT, name STRING, department STRING,
  year INT, marks INT, join_date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Upload students.csv to the lab machine, then:
LOAD DATA LOCAL INPATH '/path/students.csv'
INTO TABLE students;

-- If the CSV contains a header, remove it before loading,
-- or use a header-aware table configuration supported by your Hive version.

SELECT COUNT(*) AS total_students FROM students;
SELECT SUM(marks), AVG(marks), MIN(marks), MAX(marks) FROM students;
SELECT UPPER(name), LOWER(department), LENGTH(name) FROM students;
```

```
SELECT SUBSTR(name,1,3) FROM students;
SELECT YEAR(TO_DATE(join_date)) FROM students;
```

Illustrative output

```
total_students
5

sum(marks)  avg(marks)  min(marks)  max(marks)
411         82.2        68          92

UPPER(name)  LOWER(department)  LENGTH(name)
ASHA         cse                4
BALA         it                 4
CHARAN       cse                6
DIVYA        ece                5
ELAN         it                 4
```

Result: Hive aggregate, string and date functions are executed and their outputs are verified against the sample data.

2. Hands-on Try-out – Advanced Hive

Aim

To create partitioned and bucketed Hive tables and run filtered queries.

Procedure

7. Create or use the bigdata_demo database.
8. Create a partitioned table using department as the partition column.
9. Enable dynamic partitioning if inserting data dynamically.
10. Insert records into the partitioned table.
11. Create a bucketed table using a meaningful column such as id.
12. Run a filter query and inspect the table definitions.

Queries

```
USE bigdata_demo;

CREATE TABLE student_partitioned (
  id INT, name STRING, year INT, marks INT
)
PARTITIONED BY (department STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;

INSERT INTO TABLE student_partitioned PARTITION(department)
SELECT id, name, year, marks, department FROM students;

SHOW PARTITIONS student_partitioned;
DESCRIBE FORMATTED student_partitioned;

CREATE TABLE student_bucketed (
  id INT, name STRING, department STRING, year INT, marks INT
)
CLUSTERED BY (id) INTO 4 BUCKETS
STORED AS ORC;
```

```
SET hive.enforce.bucketing=true;
INSERT INTO TABLE student_bucketed
SELECT id, name, department, year, marks FROM students;

SELECT name, marks
FROM student_partitioned
WHERE department='CSE';
```

Illustrative output and explanation

```
name    marks
Asha    85
Charan  92

SHOW PARTITIONS may list:
department=CSE
department=ECE
department=IT
```

Partitioning stores data in separate partitions based on a column such as department. Queries filtering that column can scan fewer partitions. Bucketing distributes rows into a fixed number of files based on a hash of the bucket column; it can help with joins and sampling. Exact support and required settings depend on the Hive lab version.

3. Hands-on Try-out – Spark Core (RDD)

Aim

To process text data using Spark RDD transformations and actions and calculate word frequencies.

Program (PySpark)

```
# Save as spark_core_wordcount.py
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SparkCoreWordCount").getOrCreate()
sc = spark.sparkContext

lines = sc.parallelize([
    "big data spark",
    "spark processes big data",
    "big data"
])

words = lines.flatMap(lambda line: line.lower().split())   # transformation
pairs = words.map(lambda word: (word, 1))                  # transformation
counts = pairs.reduceByKey(lambda a, b: a + b)             # transformation

print("Word count:", words.count())                        # action
for word, count in sorted(counts.collect()):               # action
    print(word, count)

spark.stop()
```

Run command and sample output

```
spark-submit spark_core_wordcount.py

Word count: 9
big 3
data 3
```

```
processes 1
spark 2
```

Explanation: Transformations such as flatMap, map and reduceByKey create a new RDD and are evaluated lazily. Actions such as count and collect trigger execution and return a result to the driver.

Result: Word frequencies are calculated using Spark Core RDD operations.

4. DataFrame – Demos

Aim

To load a CSV file into a Spark DataFrame and perform schema inspection, selection, filtering and aggregation.

Program (PySpark)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count

spark = SparkSession.builder.appName("DataFrameDemo").getOrCreate()

df = (spark.read.option("header", True)
      .option("inferSchema", True)
      .csv("students.csv"))

df.printSchema()
df.show()

df.select("name", "marks").show()
df.filter(df.marks >= 80).show()
df.groupBy("department").agg(
    count("*").alias("student_count"),
    avg("marks").alias("average_marks")
).show()

spark.stop()
```

Illustrative output

```
root
 |-- id: integer
 |-- name: string
 |-- department: string
 |-- year: integer
 |-- marks: integer
 |-- join_date: string

+---+------+----------+----+-----+----------+
|id |name  |department|year|marks|join_date |
+---+------+----------+----+-----+----------+
|1  |Asha  |CSE       |2024|85   |2024-06-10|
|2  |Bala  |IT        |2024|78   |2024-07-15|
|3  |Charan|CSE       |2025|92   |2025-06-12|
|4  |Divya |ECE       |2025|68   |2025-07-20|
|5  |Elan  |IT        |2025|88   |2025-06-18|
+---+------+----------+----+-----+----------+

Filtered marks >= 80: Asha(85), Charan(92), Elan(88)

department  student_count  average_marks
```

```
CSE         2               88.5
ECE         1               68.0
IT          2               83.0
```

Result: The schema and DataFrame operations are displayed, and the department averages match the input data.

5. Hands-on Try-out – Spark SQL

Aim

To query a Spark DataFrame using SQL and save the query result in a supported file format.

Program (PySpark)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SparkSQLDemo").getOrCreate()

df = (spark.read.option("header", True)
      .option("inferSchema", True)
      .csv("students.csv"))

df.createOrReplaceTempView("students")

spark.sql("SELECT * FROM students").show()
spark.sql("SELECT name, marks FROM students WHERE marks >= 80").show()
spark.sql("""
    SELECT department, COUNT(*) AS total,
           ROUND(AVG(marks), 2) AS average
    FROM students
    GROUP BY department
    ORDER BY department
""").show()

result = spark.sql("SELECT name, marks FROM students WHERE marks >= 80")
result.write.mode("overwrite").option("header", True).csv("output/high_scorers")

# Read saved output again to verify it was written successfully.
saved = spark.read.option("header", True).option("inferSchema",
True).csv("output/high_scorers")
saved.show()

spark.stop()
```

Illustrative output

```
department  total  average
CSE         2      88.50
ECE         1      68.00
IT          2      83.00

Saved output can be read again and shows:
Asha   85
Charan 92
Elan   88
```

Note: Spark writes CSV output to a directory containing part files, not usually to one named CSV file. Result: SQL queries run and the saved result is read back successfully.

6. Unix Demos

Aim

To practise basic Unix/Linux file and directory commands and locate words in a text file.

Commands and procedure

```
pwd                          # show current directory
mkdir bigdata_practice        # create directory
cd bigdata_practice
printf "spark big data\nhive big data\nspark sql\n" > sample.txt
ls                           # list files
cat sample.txt               # display full file
head -n 2 sample.txt         # first two lines
tail -n 2 sample.txt         # last two lines
cp sample.txt copy.txt       # copy file
mv copy.txt renamed.txt      # rename/move file
grep -n "spark" sample.txt   # find matching lines
grep -o "big" sample.txt | wc -l  # count matching words
wc -l sample.txt             # count lines
wc -w sample.txt             # count words
cat sample.txt | grep "big" # demonstrate a pipe
```

Illustrative output

```
$ pwd
/home/student
$ grep -n "spark" sample.txt
1:spark big data
3:spark sql
$ wc -l sample.txt
3 sample.txt
$ wc -w sample.txt
8 sample.txt
$ grep -o "big" sample.txt | wc -l
2
```

Result: Directory, file, copy, move, display, search, count and pipe commands are demonstrated on a practice file.

7. HDFS Demos

Aim

To create an HDFS directory and demonstrate uploading, listing, reading, copying and deleting a sample file.

Procedure and commands

```
# Create a local sample file
echo "Big Data with Hadoop" > hdfs_sample.txt

# Create HDFS directory
hdfs dfs -mkdir -p /user/$USER/bigdata_demo

# Upload local file to HDFS
hdfs dfs -put hdfs_sample.txt /user/$USER/bigdata_demo/

# List HDFS directory
hdfs dfs -ls /user/$USER/bigdata_demo

# Display file content
```

```
hdfs dfs -cat /user/$USER/bigdata_demo/hdfs_sample.txt

# Copy file within HDFS
hdfs dfs -cp /user/$USER/bigdata_demo/hdfs_sample.txt /user/$USER/bigdata_demo/copy.txt

# Move/rename file within HDFS
hdfs dfs -mv /user/$USER/bigdata_demo/copy.txt /user/$USER/bigdata_demo/renamed.txt

# Delete a practice file
hdfs dfs -rm /user/$USER/bigdata_demo/renamed.txt

# Remove directory after checking it is empty
hdfs dfs -rmdir /user/$USER/bigdata_demo
```

Illustrative output

```
Found 1 items
-rw-r--r--   1 student supergroup 21 ... /user/student/bigdata_demo/hdfs_sample.txt

Big Data with Hadoop
```

Result: Basic HDFS operations are performed. Commands must be run in an environment connected to a Hadoop cluster; local shell commands alone do not create HDFS files.

8. MapReduce Demos – Word Count

Aim

To run a MapReduce word-count job and understand mapper and reducer stages.

Mapper (mapper.py)

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    for word in line.strip().lower().split():
        print(f"{word}\t1")
```

Reducer (reducer.py)

```
#!/usr/bin/env python3
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.rstrip("\n").split("\t", 1)
    count = int(count)

    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

Input file and Hadoop Streaming command

```
# input.txt
big data
big data hadoop
hadoop spark

# Upload input
hdfs dfs -mkdir -p /user/$USER/mr_input
hdfs dfs -put -f input.txt /user/$USER/mr_input/

# Example command; replace the jar path with the Hadoop Streaming
# JAR path installed in your lab.
hadoop jar /path/to/hadoop-streaming.jar \
  -input /user/$USER/mr_input \
  -output /user/$USER/mr_output \
  -mapper mapper.py \
  -reducer reducer.py \
  -files mapper.py,reducer.py

# View result
hdfs dfs -cat /user/$USER/mr_output/part-00000
```

Illustrative output

```
big     2
data    2
hadoop  2
spark   1
```

Explanation: The mapper reads input lines and emits (word, 1) pairs. Hadoop sorts and groups values by key. The reducer adds the values for each word and emits the final count. The Streaming JAR path and executable permissions depend on the lab setup.

Result: The MapReduce job produces the expected word frequencies.

9. Hive Demos – Basic Queries

Aim

To create a Hive database and table, load CSV data and execute SELECT, WHERE, GROUP BY and ORDER BY queries.

Procedure and commands

```
CREATE DATABASE hive_demo;
USE hive_demo;

CREATE TABLE student_marks (
  id INT,
  name STRING,
  department STRING,
  year INT,
  marks INT,
  join_date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Ensure the input file has no header row before loading.
LOAD DATA LOCAL INPATH '/path/students_no_header.csv'
```

```
INTO TABLE student_marks;

SELECT * FROM student_marks;
SELECT name, marks FROM student_marks WHERE marks > 80;
SELECT department, COUNT(*) AS total_students
FROM student_marks
GROUP BY department;
SELECT name, marks FROM student_marks ORDER BY marks DESC;
```

Illustrative output

```
WHERE marks > 80:
Asha    85
Charan  92
Elan    88

GROUP BY department:
CSE  2
ECE  1
IT   2

ORDER BY marks DESC:
Charan  92
Elan    88
Asha    85
Bala    78
Divya   68
```

Result: The Hive table is created, CSV data is loaded and basic queries are executed. Verify selected values with the original CSV file.

Practical exam checklist

- Write the aim and required software/environment for each practical.
- Show the input dataset or input text file.
- Include the commands or complete program used.
- Run the program in the approved lab environment and capture actual screenshots.
- Write the actual output; do not present illustrative output as a screenshot or verified lab result.
- For Hive, show table creation and loading commands as well as query outputs.
- For Spark, explain at least one transformation and one action.
- For HDFS and MapReduce, use the configured Hadoop cluster and the correct local JAR paths.

Important environment notes

Hive, Spark and Hadoop commands vary by installed version and lab configuration. The uploaded test-case sheet does not specify the operating system, Hadoop/Hive/Spark versions, cluster addresses or Streaming JAR location. Replace example paths with the paths supplied by your instructor. If a service is unavailable, document the exact commands and use the approved course lab, as directed in the test-case sheet.